

Casos De Prueba

Documento de Casos de Prueba — BurgerPOS

Materia: Procesos de Desarrollo de Software
Proyecto: BurgerPOS — Sistema POS para hamburgueseria
Entrega: 4 — Pruebas
Equipo: Luis David Sosa Fernandez, Natalia Guerrero Cabrera, Alfonso Mendoza Vazquez

Generalidades

Tecnologias utilizadas

Elemento	Tecnologia
Lenguaje	C# 12 / .NET 10
Framework de pruebas	xUnit 2.9
Biblioteca de aserciones	FluentAssertions 8.10
Base de datos de prueba	PostgreSQL 16 (contenedor Testcontainers)
ORM	Entity Framework Core 10 con Npgsql

Estrategia de prueba

Todas las pruebas son de tipo **integracion**: cada caso levanta un contenedor PostgreSQL 16 aislado mediante Testcontainers, aplica la migracion inicial completa y ejecuta operaciones reales contra la base de datos. Al finalizar la clase de prueba el contenedor se destruye automaticamente.

Se utiliza un `TestDatabaseFixture` compartido por coleccion (`[Collection("Database")]`) para evitar levantar multiples contenedores y garantizar aislamiento entre clases de prueba.

Requisitos previos para ejecutar

1. Docker Desktop corriendo (Engine running).
2. .NET 10 SDK instalado.
3. Desde la carpeta `Codigo/` :

Matriz de trazabilidad

Caso	Nombre resumido	RF	Regla de negocio	Tipo de operacion
CP-01	Crear categoria	RF-02	—	INSERT + SELECT
CP-02	Actualizar categoria	RF-02	—	UPDATE + SELECT
CP-03	Eliminar categoria	RF-02	—	DELETE + SELECT
CP-04	Crear producto con precio base	RF-02	RN-04	INSERT + SELECT
CP-05	Actualizar precio de producto	RF-02	RN-04	UPDATE + SELECT
CP-06	Crear insumo y verificar saldo inicial	RF-07	—	INSERT + SELECT
CP-07	Entrada de inventario incrementa saldo	RF-07	—	SELECT + UPDATE
CP-08	Merma con saldo insuficiente	RF-07	—	SELECT + excepcion
CP-09	Cobro con calculo correcto de IVA	RF-05	RN-01, RN-06	INSERT multiple
CP-10	Folios de venta consecutivos sin saltos	RF-05	RN-02	INSERT x2 + SELECT

Casos de prueba

CP-01 — Crear categoria del menu

Campo	Descripcion
ID	CP-01
Nombre	Crear categoria del menu y verificar persistencia
RF asociado	RF-02 Gestionar menu

Campo	Descripcion
Regla de negocio	N/A
Clase de prueba	<code>BurgerPOS.Tests.Catalogo.CategoriaTests</code>
Metodo	<code>CrearCategoria_GuardaEnBase</code>
Tipo	Integracion (BD real)

Precondicion:

Base de datos vacia recién migrada.

Pasos:

1. Instanciar `CatalogoService` con el contexto de prueba.
2. Llamar `CrearCategoriaAsync("Hamburguesas", 1)`.
3. Abrir un segundo contexto independiente.
4. Buscar la categoria por el `Id` devuelto.

Resultado esperado:

- El `Id` retornado no es `Guid.Empty`.
- La consulta en el segundo contexto devuelve un registro no nulo.
- El campo `Nombre` es `"Hamburguesas"`.

Resultado obtenido:

PASS

CP-02 — Actualizar nombre y orden visual de una categoria

Campo	Descripcion
ID	CP-02
Nombre	Actualizar categoria existente y verificar cambios en BD
RF asociado	RF-02 Gestionar menu
Regla de negocio	N/A
Clase de prueba	<code>BurgerPOS.Tests.Catalogo.CategoriaTests</code>
Metodo	<code>ActualizarCategoria_CambiaValores</code>
Tipo	Integracion (BD real)

Precondicion:

Existe una categoria "Bebidas" con `OrdenVisual = 2` creada previamente.

Pasos:

1. Crear la categoria con `CrearCategoriaAsync("Bebidas", 2)`.
2. Llamar `ActualizarCategoriaAsync(id, "Bebidas Frias", 3)` en un contexto nuevo.
3. Consultar la categoria en un tercer contexto por el mismo `Id`.

Resultado esperado:

- `Nombre` es "Bebidas Frias".
- `OrdenVisual` es 3.

Resultado obtenido:

PASS

CP-03 — Eliminar una categoria del menu

Campo	Descripcion
ID	CP-03
Nombre	Eliminar categoria y verificar que ya no existe en BD
RF asociado	RF-02 Gestionar menu
Regla de negocio	N/A
Clase de prueba	<code>BurgerPOS.Tests.Catalogo.CategoriaTests</code>
Metodo	<code>EliminarCategoria_YaNoExisteEnBase</code>
Tipo	Integracion (BD real)

Precondicion:

Existe una categoria "Temporales" en la base de datos.

Pasos:

1. Crear la categoria con `CrearCategoriaAsync("Temporales", 99)`.
2. Llamar `EliminarCategoriaAsync(id)` en un contexto nuevo.
3. Consultar la categoria por `Id` en un tercer contexto.

Resultado esperado:

- El resultado de la consulta es `null`.

Resultado obtenido:

PASS

CP-04 — Crear producto con precio base y categoria

Campo	Descripcion
ID	CP-04
Nombre	Crear producto vinculado a categoria y verificar atributos en BD
RF asociado	RF-02 Gestionar menu
Regla de negocio	RN-04 Precios historicos (el precio se almacena al momento de creacion)
Clase de prueba	<code>BurgerPOS.Tests.Catalogo.ProductoTests</code>
Metodo	<code>CrearProducto_GuardaPrecioCorrectamente</code>
Tipo	Integracion (BD real)

Precondicion:

Existe una categoria en la base de datos.

Pasos:

1. Crear una categoria `"Hamburguesas CP02"`.
2. Llamar `CrearProductoAsync("Doble Smash", "2 carnes", 89.00m, categoriaId)`.
3. Consultar el producto en un contexto independiente por su `Id`.

Resultado esperado:

- El `Id` del producto no es `Guid.Empty`.
- `PrecioBase` es `89.00`.
- `CategoriaId` coincide con la categoria creada.

Resultado obtenido:

PASS

CP-05 — Actualizar precio de un producto

Campo	Descripcion
ID	CP-05
Nombre	Actualizar el precio base de un producto existente
RF asociado	RF-02 Gestionar menu
Regla de negocio	RN-04 Solo el precio de nuevas ventas cambia; ventas pasadas conservan el precio historico
Clase de prueba	<code>BurgerPOS.Tests.Catalogo.ProductoTests</code>
Metodo	<code>ActualizarPrecio_ReflejaHistoricoEnNuevasVentas</code>
Tipo	Integracion (BD real)

Precondicion:

Existe un producto "Triple" con `PrecioBase = 110.00`.

Pasos:

1. Crear categoria y producto con precio inicial `110.00m`.
2. Llamar `ActualizarPrecioAsync(productoId, 125.00m)` en un contexto nuevo.
3. Consultar el producto en un tercer contexto.

Resultado esperado:

- `PrecioBase` es `125.00`.

Resultado obtenido:

PASS

CP-06 — Crear insumo y verificar saldo inicial

Campo	Descripcion
ID	CP-06
Nombre	Crear insumo de inventario con saldo inicial y nivel de alerta
RF asociado	RF-07 Gestionar inventario
Regla de negocio	N/A
Clase de prueba	<code>BurgerPOS.Tests.Inventario.InsumoTests</code>
Metodo	<code>CrearInsumo_GuardaSaldoInicial</code>

Campo	Descripcion
Tipo	Integracion (BD real)

Precondicion:

Base de datos disponible.

Pasos:

1. Llamar `CrearInsumoAsync("Carne 80/20", Kilogramos, 10m, 2m)`.
2. Consultar el insumo por `Id` en un contexto independiente.

Resultado esperado:

- `SaldoActual` es `10`.
- `NivelAlerta` es `2`.

Resultado obtenido:

PASS

CP-07 — Registrar entrada de inventario e incrementar saldo

Campo	Descripcion
ID	CP-07
Nombre	Registrar una entrada de insumo y verificar que el saldo aumenta
RF asociado	RF-07 Gestionar inventario
Regla de negocio	N/A
Clase de prueba	<code>BurgerPOS.Tests.Inventario.InsumoTests</code>
Metodo	<code>RegistrarEntrada_IncrementaSaldo</code>
Tipo	Integracion (BD real)

Precondicion:

Existe un insumo `"Pan Brioche"` con `SaldoActual = 50`.

Pasos:

1. Crear el insumo con saldo inicial `50m`.
2. Llamar `RegistrarEntradaAsync(insumoId, 20m, usuarioId)` en un contexto nuevo.
3. Consultar el insumo por `Id` en un tercer contexto.

Resultado esperado:

- `SaldoActual` es `70`.

Resultado obtenido:

PASS

CP-o8 — Merma con saldo insuficiente lanza excepcion

Campo	Descripcion
ID	CP-o8
Nombre	Intentar registrar una merma mayor al saldo disponible
RF asociado	RF-o7 Gestionar inventario
Regla de negocio	N/A (validacion de dominio: no se puede descontar mas de lo disponible)
Clase de prueba	<code>BurgerPOS.Tests.Inventario.InsumoTests</code>
Metodo	<code>RegistrarMerma_StockInsuficiente_LanzaExcepcion</code>
Tipo	Integracion (BD real)

Precondicion:

Existe un insumo `"Pepinillos"` con `SaldoActual = 0.5`.

Pasos:

1. Crear el insumo con saldo `0.5m`.
2. Intentar llamar `RegistrarMermaAsync(insumoId, 2m, Otro, usuarioId)` en un contexto nuevo.

Resultado esperado:

- Se lanza una excepcion de tipo `InvalidOperationException`.
- No se modifica el saldo en la base de datos.

Resultado obtenido:

PASS

CP-09 — Cobro de orden con calculo correcto de IVA

Campo	Descripcion
ID	CP-09
Nombre	Cobrar una orden y verificar que el IVA y el total se calculan segun RN-01 y RN-06
RF asociado	RF-05 Cobrar orden
Regla de negocio	RN-01 IVA 16% (Mexico); RN-06 Orden de calculo: subtotal - descuento = base gravable, base gravable x 0.16 = IVA, base gravable + IVA + propina = total
Clase de prueba	<code>BurgerPOS.Tests.Cobro.CobroTests</code>
Metodo	<code>CobrarOrden_CalculaIvaCorrectamente_RN01_RN06</code>
Tipo	Integracion (BD real)

Precondicion:

Existe una orden con una linea de producto a precio `$100.00`, sin descuento y sin propina. Existe un turno de caja abierto.

Pasos:

1. Preparar categoria, producto (\$100), turno y orden con una linea.
2. Llamar `CobrarAsync(ordenId, operadorId, turnoId, Efectivo, montoRecibido: 200m)`.
3. Revisar los campos de la venta retornada.

Resultado esperado:

- `Subtotal` es `100.00`.
- `Iva` es `16.00` (100 x 0.16).
- `Total` es `116.00` (100 + 16 + 0 propina).

Resultado obtenido:

PASS

CP-10 — Folios de venta consecutivos y sin saltos

Campo	Descripcion
ID	CP-10

Campo	Descripcion
Nombre	Verificar que dos ventas sucesivas reciben folios consecutivos
RF asociado	RF-05 Cobrar orden
Regla de negocio	RN-02 El numero de folio es consecutivo y sin saltos, generado por la secuencia <code>folio_venta_seq</code> de PostgreSQL
Clase de prueba	<code>BurgerPOS.Tests.Cobro.CobroTests</code>
Metodo	<code>CobrarOrden_FoliosConsecutivos_RN02</code>
Tipo	Integracion (BD real)

Precondicion:

Dos ordenes independientes listas para cobrar, cada una con su propio turno de caja.

Pasos:

1. Preparar dos ordenes con producto y turno propios.
2. Cobrar la primera orden y registrar `Folio` de `venta1`.
3. Cobrar la segunda orden y registrar `Folio` de `venta2`.

Resultado esperado:

- `venta2.Folio == venta1.Folio + 1`.

Resultado obtenido:

PASS

Resumen de ejecucion

Ejecucion realizada el 20/06/2026. Comando: `dotnet test BurgerPOS.slnx --logger "console;verbosity=normal"`

Duracion total de la suite: 29.5 segundos (incluye arranque y destruccion de contenedores Docker).

Caso	Metodo	Resultado	Tiempo
CP-01	<code>CrearCategoria_GuardaEnBase</code>	PASS	7 ms
CP-02	<code>ActualizarCategoria_CambiaValores</code>	PASS	27 ms
CP-03	<code>EliminarCategoria_YaNoExisteEnBase</code>	PASS	19 ms
CP-04	<code>CrearProducto_GuardaPrecioCorrectamente</code>	PASS	13 ms

Caso	Metodo	Resultado	Tiempo
CP-05	ActualizarPrecio_ReflejaHistoricoEnNuevasVentas	PASS	562 ms
CP-06	CrearInsumo_GuardaSaldoInicial	PASS	6 ms
CP-07	RegistrarEntrada_IncrementaSaldo	PASS	28 ms
CP-08	RegistrarMerma_StockInsuficiente_LanzaExcepcion	PASS	38 ms
CP-09	CobrarOrden_CalculaIvaCorrectamente_RN01_RN06	PASS	23 ms
CP-10	CobrarOrden_FoliosConsecutivos_RN02	PASS	266 ms

Total: 10 casos — 10 PASS — 0 FAIL

Todos los casos ejecutan contra una base de datos PostgreSQL 16 real levantada con Testcontainers.

En realidad en total se hicieron 17 tests, pero solo se colocan los 10 que pide la asignación.

The screenshot shows the Visual Studio Code interface with the following components:

- Left Sidebar (TESTING):** A list of 17 tests, all marked as passed (green checkmarks). The tests include:
 - BurgerPOS.Tests (net10.0) 1.0s
 - BurgerPOS.Tests.Catalogo 512ms
 - CategoríaTests 49ms
 - CrearCategoría_GuardaEnBase 6.0ms
 - ActualizarCategoría_CambiaValores 25ms
 - EliminarCategoría_YaNoExisteEnBase 18ms
 - ProductoTests 463ms
 - CrearProducto_GuardaPrecioCorrectamente 11ms
 - ActualizarPrecio_ReflejaHistoricoEnNuevasVentas 4...
 - BurgerPOS.Tests.Cobro 360ms
 - CobroTests 360ms
 - CobrarOrden_GeneraVentaConFolio 22ms
 - CobrarOrden_CalculaIvaCorrectamente_RN01_RN06...
 - CobrarOrden_FoliosConsecutivos_RN02 244ms
 - CobrarOrden_MarcaOrdenComoCobrada 28ms
 - AnularVenta_SinRolAdmin_LanzaExcepcion_RN03 3...
 - AbrirTurno_GuardaFondolIncial 12ms
 - BurgerPOS.Tests.Inventario 109ms
 - InsumoTests 109ms
 - CrearInsumo_GuardaSaldoInicial 8.0ms
 - RegistrarEntrada_IncrementaSaldo 31ms
 - RegistrarMerma_DecrementaSaldo 31ms
 - RegistrarMerma_StockInsuficiente_LanzaExcepcion...
 - BurgerPOS.Tests.Ordenes 23ms
 - CrearOrden_EstadolnicialEsAbierta 5.0ms
 - AgregarLinea_ActualizaSubtotalDeLaOrden 18ms
- Main Editor:** Displays the `TestDatabaseFixture.cs` file. The code includes:


```

1 using BurgerPOS.Infrastructure.Persistence;
2 using Microsoft.EntityFrameworkCore;
3 using Testcontainers.PostgreSql;
4
5 namespace BurgerPOS.Tests.Fixtures;
6
7 public class TestDatabaseFixture : IAsyncLifetime
8 {
9     private readonly PostgreSQLContainer _container = new PostgreSQLBuilder("postgres:16-alpine")
10         .WithDatabase("burgerpos_test")
11         .WithUsername("burgerpos")
12         .WithPassword("test")
13         .Build();
14
15     public BurgerPosDbContext CreateContext()
16     {
17         var options = new DbContextOptionsBuilder<BurgerPosDbContext>()
18             .UseNpgsql(_container.GetConnectionString())
19             .UseSnakeCaseNamingConvention()
20             .Options;
21         return new BurgerPosDbContext(options);
22     }
23
24     public async Task InitializeAsync()
25     {
26         // ...
27     }
28 }
```
- Bottom Panel (TEST RESULTS):**
 - Summary:** Total Tests: 17, Passed: 17, Failed: 0, Skipped: 0. Result: SUCCEEDED. Total Time (including build): 27.62s (27618ms).
 - Test Results List:**
 - CrearOrden_EstadolnicialEsAbierta OrdenTests - BurgerPOS.Tests.Ordenes - BurgerPOS.Tests (net10.0)
 - AgregarLinea_ActualizaSubtotalDeLaOrden OrdenTests - BurgerPOS.Tests.Ordenes - BurgerPOS.Tests (net10.0)
 - CrearInsumo_GuardaSaldoInicial InsumoTests - BurgerPOS.Tests.Inventario - BurgerPOS.Tests (net10.0)
 - RegistrarEntrada_IncrementaSaldo InsumoTests - BurgerPOS.Tests.Inventario - BurgerPOS.Tests (net10.0)
 - RegistrarMerma_DecrementaSaldo InsumoTests - BurgerPOS.Tests.Inventario - BurgerPOS.Tests (net10.0)
 - RegistrarMerma_StockInsuficiente_LanzaExcepcion InsumoTests - BurgerPOS.Tests.Inventario - BurgerPOS.Tests (net10.0)

captura de pantalla de los tests completados en VSCode